



MODUL PERKULIAHAN

SISTEM KENDALI CERDAS

Kode Mata kuliah W132500026

Modul 9

Analisis dan Perancangan Kontrol PID

Abstrak

PID tetap menjadi controller industri paling luas karena transparan, murah, dan efektif. P memperkuat koreksi saat ini, I menghapus bias masa lalu, dan D mengantisipasi perubahan—tetapi derivative harus difilter dan integral perlu anti-windup.

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 4.2 — Memahami aksi P, I, D dan melakukan tuning PID yang aman



Modul

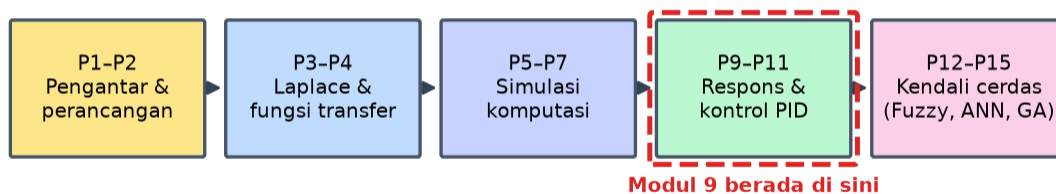
Interaktif:

<https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Sistem-Kendali-Cerdas/Modul/Modul-9.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk membaca seluruh materi tanpa perlu NIM dan PIN. Untuk mengerjakan Tugas dan Forum, masuk sebagai Mahasiswa memakai NIM dan PIN.

PENDAHULUAN

PID tetap menjadi controller industri paling luas karena transparan, murah, dan efektif. P memperkuat koreksi saat ini, I menghapus bias masa lalu, dan D mengantisipasi perubahan —tetapi derivative harus difilter dan integral perlu anti-windup.

Alur Mata Kuliah Sistem Kendali Cerdas



Gambar 1. Posisi Pertemuan 10 (Analisis dan Perancangan Kontrol PID) dalam alur mata kuliah Sistem Kendali Cerdas.

Modul ini disusun berlapis: pembahasan konsep, penurunan rumus yang dipakai, satu contoh terselesaikan, catatan salah kaprah yang sering terjadi, daftar periksa, lalu tugas terstruktur berupa sepuluh soal pilihan ganda dan lima belas soal komputasi. Versi interaktifnya menilai jawaban secara otomatis di server, sedangkan berkas ini dipakai untuk membaca dan mengerjakan secara luring.

Capaian Pembelajaran (Sub-CPMK)

- Sub-CPMK 4.2 — Memahami aksi P, I, D dan melakukan tuning PID yang aman

TIGA AKSI, TIGA PERTANYAAN BERBEDA

Controller PID menjumlahkan tiga aksi yang bekerja atas error yang sama namun menjawab pertanyaan berbeda. Aksi proporsional menjawab seberapa besar error saat ini. Aksi integral menjawab seberapa lama error itu bertahan. Aksi turunan menjawab ke arah mana error sedang bergerak.

Aksi proporsional memberi respons segera yang sebanding dengan error. Menaikkannya mempercepat sistem dan mengecilkan error tunak, namun memperbesar overshoot dan

mendekatkan sistem ke batas kestabilan. Aksi ini sendirian tidak dapat menghapus error tunak pada plant tanpa integrator.

Aksi integral mengakumulasi error terhadap waktu, sehingga selama masih ada error sekecil apa pun, keluarannya terus bertambah. Sifat inilah yang menghapus error tunak. Harganya adalah tambahan satu pole di titik asal yang menurunkan redaman dan memperlambat sistem, ditambah risiko penumpukan saat actuator jenuh.

Aksi turunan bereaksi terhadap laju perubahan error, sehingga ia mulai menahan sebelum keluaran melewati setpoint. Aksi ini menambah redaman dan memungkinkan penguatan proporsional yang lebih besar dipakai dengan aman. Kelemahannya ia memperkuat derau, sehingga hampir selalu memerlukan penapis.

$$u = K_p * e + K_i * \text{integral}(e) + K_d * de/dt \quad | \quad \text{sekarang, masa lalu, dan arah}$$

Bentuk Baku dan Arti Fisik Parameternya

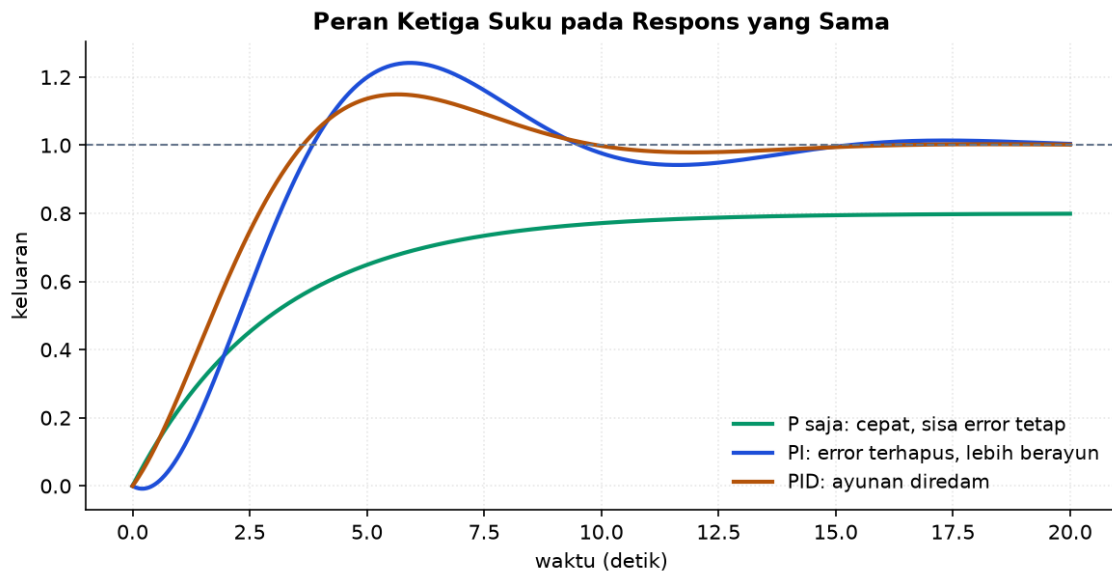
Selain bentuk penjumlahan tiga gain, PID sering dinyatakan dalam bentuk baku yang memakai waktu integral dan waktu turunan. Bentuk ini lebih disukai di dunia proses karena kedua parameternya bersatuan waktu sehingga dapat dibandingkan langsung dengan dinamika plant.

Waktu integral menyatakan berapa lama aksi integral memerlukan waktu untuk menyumbang sebesar sumbangan aksi proporsional pada error tetap. Nilai yang kecil berarti aksi integral agresif. Aturan kasar yang lazim menetapkan sebanding dengan konstanta waktu dominan plant.

Waktu turunan menyatakan seberapa jauh ke depan controller memperkirakan gerak error. Nilai yang terlalu besar membuat controller bereaksi berlebihan terhadap perubahan kecil, dan bersama derau sensor menghasilkan getaran pada actuator. Praktik yang lazim menetapkan sekitar seperempat waktu integral.

Perlu diperhatikan bahwa mengubah penguatan proporsional pada bentuk baku ikut mengubah kekuatan aksi integral dan turunan, karena keduanya dikalikan penguatan yang sama. Pada bentuk penjumlahan tiga gain, ketiganya bebas satu sama lain. Kekeliruan menyamakan kedua bentuk adalah sumber kebingungan yang sering terjadi saat memindahkan parameter antarperangkat.

$$u = K_p * (e + (1/T_i) * \text{integral}(e) + T_d * de/dt) \quad | \quad T_i \text{ dan } T_d \text{ bersatuan detik}$$



Gambar 2. Pengaruh penambahan suku integral dan turunan terhadap respons controller proporsional.

Gambar 2 merangkum pembagian tugas ketiga suku: proporsional menangani besar error, integral menghapus sisanya, dan turunan meredam ayunan yang ditimbulkan integral.

Penyetelan: Dari Aturan Praktis ke Penyetelan Berbasis Model

Metode Ziegler-Nichols memberi titik awal dari dua percobaan sederhana. Cara pertama menaikkan penguatan proporsional sampai sistem berosilasi dengan amplitudo tetap, lalu mencatat penguatan kritis dan periode osilasinya. Cara kedua memakai respons step lintasan terbuka untuk menaksir gain, konstanta waktu, dan dead time.

Yang perlu disadari, aturan ini dirancang untuk mengejar peredaman seperempat amplitudo, yang menghasilkan respons cukup agresif dengan overshoot sekitar dua puluh sampai lima puluh persen. Untuk banyak penerapan modern hasil itu terlalu berayun, sehingga aturan ini lebih tepat dipakai sebagai titik awal daripada nilai akhir.

Penyetelan berbasis model bekerja dari arah berlawanan. Spesifikasi diterjemahkan menjadi letak pole yang dituju, lalu parameter controller dihitung agar pole loop tertutup mendarat di sana. Cara ini menuntut model yang cukup baik, namun memberi kendali langsung atas overshoot dan waktu menetap.

Apa pun metodenya, penyetelan tidak berhenti di simulasi. Nilai hasil hitungan dipakai sebagai titik awal, lalu diperiksa di perangkat dengan menaikkan agresivitas bertahap sambil mengamati sinyal kendali. Yang menghentikan proses bukan grafik yang indah, melainkan tercapainya spesifikasi tanpa menjenuhkan actuator dan tanpa memperkuat derau berlebihan.

$$\text{Ziegler-Nichols: } K_p = 0,6 \cdot K_u, \quad T_i = 0,5 \cdot T_u, \quad T_d = 0,125 \cdot T_u$$

Anti-Windup dan Pembatasan yang Wajib Ada

Ketika actuator mencapai batasnya, keluaran controller tidak lagi memengaruhi plant. Aksi integral yang tetap menumpuk selama itu akan membesar tanpa guna, dan ketika error akhirnya berbalik tanda, akumulator yang telanjur besar harus dikosongkan lebih dahulu sebelum keluaran turun. Akibatnya keluaran melewati setpoint jauh dan lama.

Gejalanya khas dan mudah dikenali: overshoot yang jauh lebih besar daripada perkiraan simulasi, terutama setelah perubahan setpoint yang besar, disertai keluaran yang bertahan mentok cukup lama. Bila gejala ini muncul, memeriksa anti-windup jauh lebih tepat daripada menurunkan penguatan.

Penanganan paling sederhana menghentikan penumpukan ketika keluaran sedang mentok dan error masih mendorong ke arah yang sama. Cara yang lebih halus mengumpat-balikkan selisih antara keluaran yang diminta dan yang benar-benar terjadi ke akumulator, sehingga akumulator menyesuaikan diri secara mulus alih-alih dibekukan.

Selain kejenuhan, laju perubahan actuator juga sering terbatas. Katup memerlukan waktu untuk bergerak, dan motor memiliki batas percepatan. Pembatasan laju ini menimbulkan gejala serupa dan perlu dimodelkan bila dinamikanya sebanding dengan dinamika loop.

anti-windup: $I += Kt*(u_nyata - u_minta)*T$ bersama pembatasan keluaran

Varian Struktur yang Sering Diperlukan

PID dasar sering perlu disesuaikan agar berperilaku baik di lapangan. Salah satu penyesuaian yang paling umum adalah menghitung aksi turunan dari keluaran terukur, bukan dari error. Dengan cara ini, perubahan setpoint yang mendadak tidak lagi menghasilkan lonjakan turunan yang besar, sementara kemampuan meredam tetap terjaga.

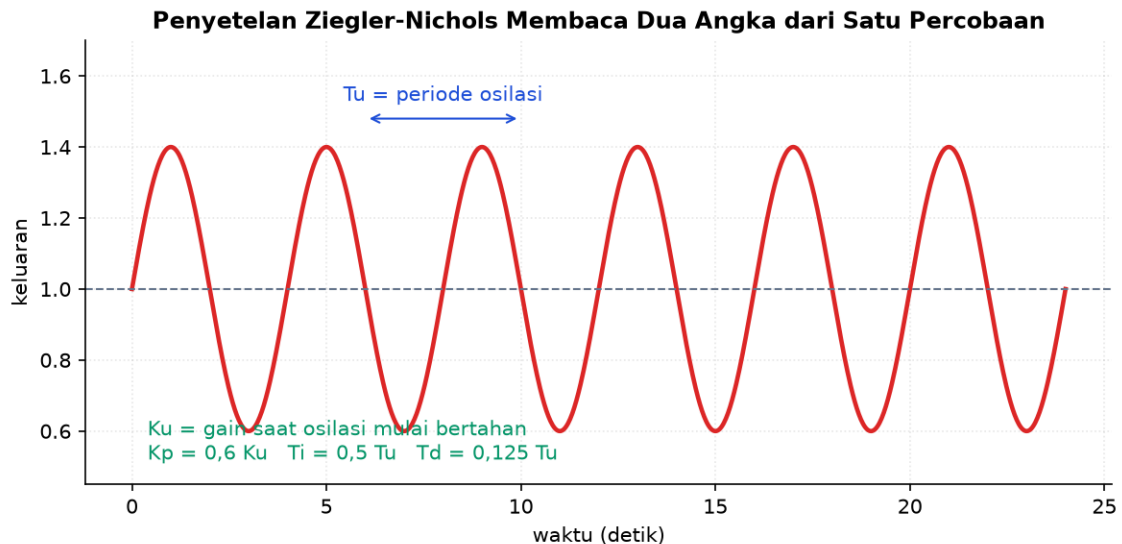
Pembobotan setpoint pada aksi proporsional bekerja dengan semangat serupa. Aksi proporsional dihitung dari selisih antara setpoint berbobot dan keluaran, sehingga respons terhadap perubahan setpoint dapat dilunakkan tanpa mengubah sifat penolakan gangguan maupun kestabilan loop.

Untuk proses berdead-time besar, struktur prediktor memisahkan pengaruh tundaan dari perhitungan umpan balik dengan memakai model plant. Sistem menjadi jauh lebih agresif secara aman, namun kinerjanya kini bergantung pada ketepatan model. Bila model meleset, keunggulannya cepat hilang.

Pada sistem dengan besaran cepat di dalam besaran lambat, struktur bersarang biasanya lebih efektif daripada satu PID tunggal yang disetel sangat agresif. Loop dalam menekan

gangguan di dekat sumbernya dan menyederhanakan penyetelan, karena tiap loop dapat disetel bergiliran dari dalam ke luar.

turunan dari keluaran: $D = -K_d \cdot dy/dt$ | bobot setpoint: $P = K_p \cdot (b \cdot r - y)$



Gambar 3. Uji osilasi berkelanjutan yang menghasilkan gain kritis dan periode kritis.

Gambar 3 menjelaskan mengapa metode ini bertahan lama: dua angka yang dibaca dari satu percobaan sudah cukup untuk memberi titik awal penyetelan yang wajar.

Menyetel di Lapangan dengan Urutan yang Aman

Penyetelan di perangkat menuntut urutan yang berbeda dari perancangan di atas kertas, karena setiap langkah membawa risiko nyata. Prinsipnya bergerak dari yang paling aman menuju yang paling agresif, dengan jalan keluar yang selalu siap.

Mulai dengan aksi proporsional saja, aksi integral dan turunan dinolkan. Naikkan penguatan bertahap sambil mengamati respons terhadap perubahan setpoint kecil, sampai muncul osilasi ringan yang mereda. Catat nilai itu; ia menjadi acuan untuk seluruh langkah berikutnya.

Baru kemudian aksi integral ditambahkan, dimulai dari waktu integral yang besar lalu diperkecil bertahap. Waktu integral yang terlalu kecil akan menampilkan diri sebagai ayunan lambat yang berkepanjangan, dan itulah tanda untuk mundur satu langkah.

Aksi turunan ditambahkan terakhir, dan hanya bila diperlukan. Ia dinaikkan sampai peredaman membaik namun berhenti sebelum aktuator mulai bergetar. Bila getaran muncul sebelum peredaman terasa membaik, penapisnya yang perlu diperbaiki, bukan penguatannya yang diturunkan.

P dulu -> tambahkan I dari Ti besar -> tambahkan D terakhir bila perlu

Loop Bersarang: Menyetel dari Dalam ke Luar

Ketika besaran cepat berada di dalam besaran lambat, struktur bersarang hampir selalu mengungguli satu PID tunggal. Contohnya arus di dalam kecepatan, kecepatan di dalam posisi, atau aliran di dalam temperatur.

Keunggulannya berasal dari dua hal. Loop dalam menekan gangguan di dekat sumbernya sebelum sempat memengaruhi besaran luar, dan loop dalam juga meratakan nonlinieritas aktuator sehingga loop luar menghadapi perilaku yang lebih sederhana.

Penyetelannya dilakukan bergiliran dari dalam ke luar. Loop dalam disetel lebih dahulu dengan loop luar dalam mode manual, dan disetel agar jauh lebih cepat daripada loop luar, umumnya lima sampai sepuluh kali. Setelah loop dalam mantap, barulah loop luar disetel dengan memperlakukan loop dalam sebagai penguat sederhana.

Kesalahan yang paling sering terjadi adalah menyetel keduanya bersamaan atau menyetel loop luar lebih dahulu. Keduanya menghasilkan interaksi yang membingungkan, karena perubahan pada satu loop mengubah perilaku yang sedang diamati pada loop lain.

loop dalam ≥ 5 kali lebih cepat, disetel lebih dahulu dengan loop luar manual

PID pada Proses Berdead-Time Besar

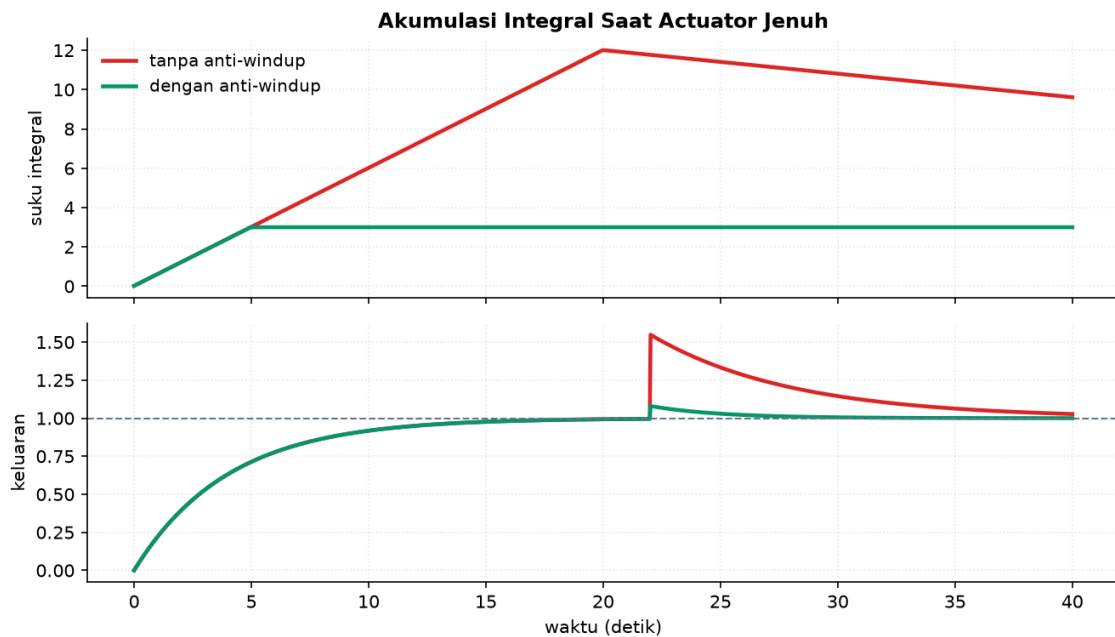
Dead time adalah lawan paling berat bagi PID. Selama jeda itu, controller tidak menerima kabar apa pun tentang akibat tindakannya, sehingga ia cenderung bertindak berlebihan dan baru menyadarinya setelah terlambat.

Akibatnya terhadap penyetelan bersifat mendasar. Semakin besar perbandingan dead time terhadap konstanta waktu, semakin kecil penguatan yang masih aman. Pada perbandingan di atas satu, PID biasa hanya dapat disetel sangat lamban, dan kinerjanya terbatas berapa pun keterampilan penyetelnya.

Langkah pertama yang benar bukan menyetel melainkan memeriksa apakah dead time-nya dapat dikurangi. Memindahkan sensor lebih dekat ke titik yang dikendalikan, memperpendek jalur perpipaan, atau mempercepat pencuplikan sering memberi perbaikan yang jauh melampaui hasil penyetelan mana pun.

Bila dead time memang tidak dapat dikurangi, struktur prediktor memakai model plant untuk memperkirakan akibat tindakan sebelum akibat itu terukur. Sistem menjadi jauh lebih agresif secara aman, dengan syarat modelnya cukup tepat — dan bila model meleset, keunggulannya cepat hilang.

L/τ besar $\Rightarrow$ penguatan aman mengecil; kurangi L dulu sebelum menyetel



Gambar 4. Windup integral dan akibatnya pada keluaran, dibandingkan dengan controller yang memakai anti-windup.

Gambar 4 memperlihatkan satu kesalahan yang paling sering muncul di lapangan: suku integral terus menumpuk selama actuator jenuh, lalu tumpukan itu harus dibayar dengan lonjakan setelahnya.

Mengenali Ketika PID Bukan Jawabannya

PID menyelesaikan sebagian besar persoalan kendali di industri, namun ada keadaan tempat ia bukan pilihan yang tepat, dan mengenalinya lebih awal menghemat banyak waktu.

Keadaan pertama adalah sistem yang tuntutan perilakunya berbeda di daerah operasi berbeda. Satu himpunan parameter tidak dapat sekaligus lembut di sekitar setpoint dan agresif saat error besar; yang dibutuhkan adalah penjadwalan gain atau controller yang hubungannya tidak linier.

Keadaan kedua adalah sistem dengan banyak masukan dan keluaran yang saling berinteraksi kuat. Menyetel beberapa PID terpisah pada sistem semacam itu menghasilkan loop yang saling mengganggu, dan penanganannya menuntut perancangan yang memandang seluruh sistem sekaligus.

Keadaan ketiga adalah sistem dengan batasan yang harus dihormati secara eksplisit, misalnya tekanan yang tidak boleh terlampaui sambil tetap mengejar hasil maksimum. PID tidak memiliki cara menyatakan batasan; yang dibutuhkan adalah metode yang mengoptimalkan sambil memperhitungkan batas itu secara langsung.

perilaku beda per daerah, interaksi antarloop kuat, atau batasan eksplisit

MENURUNKAN PARAMETER PI DARI LETAK POLE YANG DITUJU

Penurunan berikut menunjukkan cara menghitung parameter controller langsung dari spesifikasi, bukan lewat coba-coba.

1. Plant dan controller

$$G = K/(\tau s + 1), \quad C = K_p + K_i/s$$

Dipilih PI karena spesifikasi menuntut error tunak nol terhadap step.

2. Gain lintasan terbuka

$$L = (K_p s + K_i) K / (s (\tau s + 1))$$

Controller disatukan menjadi satu pecahan.

3. Persamaan karakteristik

$$\tau s^2 + (1 + K K_p) s + K K_i = 0$$

Diperoleh dari $1 + L = 0$ lalu dikalikan penyebutnya.

4. Bentuk baku orde dua

$$s^2 + (1 + K K_p)/\tau s + K K_i/\tau = 0$$

Seluruh persamaan dibagi tau.

5. Cocokkan suku konstanta

$$\omega_n^2 = K K_i/\tau \Rightarrow K_i = \tau \omega_n^2 / K$$

Frekuensi alami menentukan K_i .

6. Cocokkan koefisien s

$$2 \zeta \omega_n = (1 + K K_p)/\tau \Rightarrow K_p = (2 \zeta \omega_n \tau - 1)/K$$

Rasio redaman menentukan K_p .

Kedua parameter kini diperoleh langsung dari ζ dan ω_n , yang sendiri diturunkan dari batas overshoot dan waktu menetap. Perhatikan K_p dapat bernilai negatif bila ω_n yang diminta terlalu kecil — pertanda spesifikasinya sendiri tidak masuk akal untuk plant tersebut.

CONTOH TERHITUNG: MERANCANG PI DARI SPESIFIKASI

Diketahui: Plant $G(s) = 2/(5s + 1)$, yaitu $K = 2$ dan $\tau = 5$ detik · Spesifikasi: overshoot maksimum 10 persen dan waktu menetap maksimum 4 detik pada pita dua persen

1. Rasio redaman dari overshoot

$$z = -\ln(0,10)/\sqrt{\pi^2 + \ln(0,10)^2} = 0,5912$$

Batas overshoot menetapkan z.

2. Frekuensi alami dari waktu menetap

$$\omega_n = 4/(z \cdot t_s) = 4/(0,5912 \cdot 4) = 1,6917$$

Tuntutan kecepatan menetapkan ω_n .

3. Hitung K_i

$$K_i = \tau \cdot \omega_n^2 / K = 5 \cdot 2,8619 / 2 = 7,1547$$

Memakai hasil pencocokan suku konstanta.

4. Hitung K_p

$$K_p = (2 \cdot z \cdot \omega_n \cdot \tau - 1) / K = (2 \cdot 0,5912 \cdot 1,6917 \cdot 5 - 1) / 2$$

Memakai hasil pencocokan koefisien s.

5. Selesaikan K_p

$$= (10,0 - 1) / 2 = 4,5$$

Perhatikan $2 \cdot z \cdot \omega_n = 4/t_s = 1$, sehingga $2 \cdot z \cdot \omega_n \cdot \tau = 5$.

6. Periksa waktu integral

$$T_i = K_p / K_i = 4,5 / 7,1547 = 0,629 \text{ s}$$

Jauh lebih kecil daripada $\tau = 5 \text{ s}$, artinya aksi integral memang agresif.

Jawaban. Diperoleh $K_p = 4,5$ dan $K_i = 7,1547$. Nilai ini titik awal, bukan nilai akhir: sebelum dipakai, sinyal kendali pada saat awal harus diperiksa terhadap batas actuator, dan aksi integral yang agresif menuntut anti-windup yang benar-benar bekerja.

SALAH KAPRAH YANG SERING TERJADI

- Menambah aksi integral tanpa anti-windup — Actuator mentok adalah kejadian biasa. Tanpa penanganan, keluaran melewati setpoint jauh melebihi perkiraan simulasi.
- Memakai aksi turunan tanpa penapis — Selisih memperkuat derau sensor sehingga actuator bergetar terus-menerus dan cepat aus.
- Menyamakan bentuk baku dengan bentuk tiga gain — Pada bentuk baku, mengubah K_p ikut mengubah kekuatan integral dan turunan. Memindahkan parameter antarperangkat tanpa memeriksa bentuknya menghasilkan perilaku yang jauh berbeda.

- Memperlakukan Ziegler-Nichols sebagai nilai akhir — Aturan itu mengejar peredaman seperempat amplitudo yang menghasilkan overshoot dua puluh sampai lima puluh persen — terlalu agresif untuk banyak penerapan.
- Menghitung turunan dari error saat setpoint berubah mendadak — Lonjakan turunan yang besar mengagetkan actuator. Menghitungnya dari keluaran terukur menghilangkan gejala itu tanpa mengurangi kemampuan meredam.

DAFTAR PERIKSA SEBELUM LANJUT

- ☐ Kebutuhan aksi integral ditentukan dari tipe sistem, bukan dari selera
- ☐ Bentuk PID yang dipakai perangkat diperiksa sebelum memindahkan parameter
- ☐ Parameter awal diturunkan dari spesifikasi lewat z dan wn , bukan coba-coba
- ☐ Anti-windup diterapkan dan diuji pada keadaan actuator mentok
- ☐ Aksi turunan dilengkapi penapis dan dihitung dari keluaran bila setpoint sering berubah
- ☐ Sinyal kendali diperiksa terhadap batas actuator sebelum penerapan
- ☐ Penyetelan akhir diverifikasi lewat pengujian setpoint dan gangguan

TUGAS

Tugas dikerjakan pada halaman modul interaktif agar penilaiannya otomatis. Bobotnya 50 poin: sepuluh soal pilihan ganda @1 poin, sepuluh soal komputasi tingkat dasar-menengah @2 poin, dan lima soal komputasi tingkat lanjut @4 poin. Soal komputasi dikerjakan dengan Python; yang dinilai adalah nilai yang dicetak, bukan cara penulisan programnya.

A. Pilihan Ganda (10 soal @1 poin)

1. Aksi integral menjawab pertanyaan...

- A. Seberapa besar error saat ini
- B. Seberapa lama error itu bertahan
- C. Ke arah mana error bergerak
- D. Seberapa besar derau sensor

2. Perbedaan pokok bentuk baku PID terhadap bentuk tiga gain adalah...

- A. Bentuk baku tidak memiliki aksi turunan
- B. Pada bentuk baku, mengubah K_p ikut mengubah kekuatan aksi integral dan turunan

- C. Bentuk baku hanya berlaku untuk sistem digital
 - D. Keduanya identik dalam segala hal
3. Aturan Ziegler-Nichols dirancang untuk mengejar...
- A. Overshoot nol
 - B. Peredaman seperempat amplitudo
 - C. Error tunak minimum
 - D. Margin fase 60 derajat
4. Gejala khas penumpukan integral saat actuator jenuh adalah...
- A. Getaran rapat beramplitudo kecil
 - B. Overshoot jauh lebih besar daripada perkiraan simulasi, disertai keluaran yang lama bertahan mentok
 - C. Error tunak yang membesar
 - D. Respons yang tidak pernah bergerak
5. Menghitung aksi turunan dari keluaran terukur alih-alih dari error bertujuan...
- A. Menghemat waktu komputasi
 - B. Menghilangkan kebutuhan penapis
 - C. Menghindari lonjakan turunan saat setpoint berubah mendadak
 - D. Menghapus error tunak
6. Waktu turunan yang terlalu besar menimbulkan...
- A. Error tunak yang membesar
 - B. Reaksi berlebihan terhadap perubahan kecil dan getaran pada actuator
 - C. Respons yang sangat lambat
 - D. Hilangnya kemampuan menolak gangguan
7. Perancangan berbasis model bekerja dengan cara...
- A. Menaikkan gain sampai berosilasi lalu menerapkan tabel
 - B. Menerjemahkan spesifikasi menjadi letak pole lalu menghitung parameter agar pole tertutup mendarat di sana
 - C. Mencoba nilai acak sampai respons memuaskan

- D. Menyalin parameter dari loop lain yang mirip
8. Pada struktur bersarang, loop dalam dirancang jauh lebih cepat agar...
- A. Loop luar dapat memperlakukannya sebagai penguat sederhana
 - B. Sensor dapat dipakai bersama
 - C. Aksi integral tidak diperlukan
 - D. Waktu cacah dapat diperbesar
9. Struktur prediktor untuk proses berdead-time besar memiliki kelemahan berupa...
- A. Tidak dapat memakai aksi integral
 - B. Kinerjanya bergantung pada ketepatan model plant
 - C. Menuntut waktu cacah sangat kecil
 - D. Hanya berlaku untuk sistem orde satu
10. Nilai parameter hasil perhitungan sebaiknya diperlakukan sebagai...
- A. Nilai akhir yang langsung dipasang
 - B. Titik awal yang diperiksa terhadap batas actuator lalu diuji bertahap
 - C. Perkiraan kasar yang boleh diabaikan
 - D. Nilai yang hanya berlaku di simulasi

B. Komputasi Dasar-Menengah (10 soal @2 poin)

Parameter acuan: Plant orde satu $G(s) = 3/(4s + 1)$ dengan umpan balik satuan. Spesifikasi rancangan: overshoot maksimum 15 persen dan waktu menetap maksimum 3 detik pada pita dua persen. Untuk pembandingan, uji penguatan kritis memberi $K_u = 8$ dengan periode osilasi $T_u = 2$ detik. Actuator terbatas pada keluaran 5 satuan.

Tabel 1. Parameter acuan yang dipakai seluruh soal komputasi modul ini.

Parameter	Nilai
K / tau	3 / 4 s
M_p / t_s	15 % / 3 s
K_u / T_u	8 / 2 s
batas actuator	5 satuan

C1. Dari batas overshoot 15 persen, hitung rasio redaman yang diperlukan. Cetak: z.

– PETUNJUK: Langkah: 1) tulis $M_p = 0,15$ sebagai pecahan. 2) hitung $\ln(M_p)$. 3) hitung $\sqrt{\pi^2 + \ln(M_p)^2}$. 4) $z = -\ln(M_p)$ dibagi hasil itu.

C2. Dari batas waktu menetap 3 detik, hitung frekuensi alami yang diperlukan, dalam rad/s. Cetak: ω_n .

– PETUNJUK: Langkah: 1) tulis $t_s = 4/(z \cdot \omega_n)$. 2) peroleh hasil kali $z \cdot \omega_n = 4/t_s$. 3) peroleh z dari soal sebelumnya. 4) $\omega_n = (z \cdot \omega_n)/z$.

C3. Hitung K_p rancangan berbasis spesifikasi. Cetak: K_p .

– PETUNJUK: Langkah: 1) peroleh z dan ω_n . 2) hitung hasil kali $2 \cdot z \cdot \omega_n$. 3) kalikan dengan τ . 4) kurangi satu lalu bagi dengan K .

C4. Hitung K_i rancangan berbasis spesifikasi. Cetak: K_i .

– PETUNJUK: Langkah: 1) peroleh ω_n . 2) kuadratkan ω_n . 3) kalikan dengan τ . 4) bagi dengan K .

C5. Hitung waktu integral T_i rancangan, dalam detik. Cetak: T_i .

– PETUNJUK: Langkah: 1) peroleh K_p . 2) peroleh K_i . 3) $T_i = K_p/K_i$.

C6. Hitung keluaran aksi proporsional pada saat awal untuk perubahan setpoint satu satuan. Cetak: $u_P(0)$.

– PETUNJUK: Langkah: 1) pada saat awal seluruh setpoint menjadi error. 2) peroleh K_p . 3) $u_P(0) = K_p \times \text{error awal}$.

C7. Hitung bagian nyata pole loop tertutup rancangan. Cetak: $\text{Re}(s)$.

– PETUNJUK: Langkah: 1) peroleh z dan ω_n . 2) bagian nyata = $-z \cdot \omega_n$. 3) periksa hasilnya sama dengan $-4/t_s$.

C8. Hitung bagian imajiner pole loop tertutup rancangan. Cetak: $\text{Im}(s)$.

– PETUNJUK: Langkah: 1) peroleh z . 2) peroleh ω_n . 3) hitung $\sqrt{1 - z^2}$. 4) $\text{Im} = \omega_n \times \sqrt{1 - z^2}$.

C9. Hitung waktu puncak rancangan, dalam detik. Cetak: t_p .

– PETUNJUK: Langkah: 1) peroleh z dan ω_n . 2) hitung $\omega_d = \omega_n \cdot \sqrt{1 - z^2}$. 3) $t_p = \pi/\omega_d$.

C10. Dari uji penguatan kritis, hitung K_p menurut aturan Ziegler-Nichols untuk PID. Cetak: $K_p \text{ ZN}$.

– PETUNJUK: Langkah: 1) ambil K_u dari parameter acuan. 2) aturan PID memakai $K_p = 0,6 \times K_u$. 3) hitung nilainya.

C. Komputasi Lanjut (5 soal @4 poin)

C11 (Lanjut). Hitung K_d menurut aturan Ziegler-Nichols untuk PID. Cetak: $K_d \text{ ZN}$.

- PETUNJUK: Langkah: 1) ambil K_u dan T_u . 2) hitung $K_p = 0,6 \cdot K_u$. 3) hitung $T_i = 0,5 \cdot T_u$. 4) hitung $T_d = 0,125 \cdot T_u$. 5) kenali $K_d = K_p \cdot T_d$ pada bentuk tiga gain. 6) hitung nilainya.

C12 (Lanjut). Hitung rasio K_p aturan Ziegler-Nichols terhadap K_p rancangan berbasis spesifikasi. Cetak: rasio K_p .

- PETUNJUK: Langkah: 1) peroleh z dari batas overshoot. 2) peroleh w_n dari batas waktu menetap. 3) hitung K_p rancangan. 4) hitung $K_p \text{ ZN} = 0,6 \cdot K_u$. 5) bagi $K_p \text{ ZN}$ dengan K_p rancangan. 6) tafsirkan seberapa jauh lebih agresif aturan praktis itu.

C13 (Lanjut). Setpoint berubah 3 satuan dan penyetelan Ziegler-Nichols dipakai. Hitung kelebihan keluaran terhadap batas actuator pada saat awal. Cetak: kelebihan.

- PETUNJUK: Langkah: 1) hitung $K_p \text{ ZN} = 0,6 \cdot K_u$. 2) pada saat awal seluruh perubahan setpoint menjadi error. 3) hitung $u(0) = K_p \times 3$. 4) ambil batas actuator 5 satuan. 5) kurangkan batas dari $u(0)$. 6) simpulkan apakah actuator jenuh.

C14 (Lanjut). Hitung rasio waktu integral Ziegler-Nichols terhadap waktu integral rancangan. Cetak: rasio T_i .

- PETUNJUK: Langkah: 1) hitung K_p dan K_i rancangan dari z dan w_n . 2) hitung $T_i \text{ rancangan} = K_p / K_i$. 3) hitung T_i menurut Ziegler-Nichols $= 0,5 \cdot T_u$. 4) bagi $T_i \text{ ZN}$ dengan $T_i \text{ rancangan}$. 5) hitung nilainya. 6) tafsirkan mana yang aksi integralnya lebih agresif.

C15 (Lanjut). Verifikasi rancangan: dari K_p dan K_i yang diperoleh, hitung kembali rasio redaman loop tertutup lewat persamaan karakteristik. Cetak: z hasil verifikasi.

- PETUNJUK: Langkah: 1) hitung K_p dan K_i rancangan. 2) susun persamaan karakteristik $\tau s^2 + (1 + K \cdot K_p)s + K \cdot K_i = 0$. 3) bagi seluruhnya dengan τ . 4) $w_n = \text{akar dari suku konstanta}$. 5) koefisien s sama dengan $2 \cdot z \cdot w_n$. 6) selesaikan z lalu bandingkan dengan target awal.

DAFTAR PUSTAKA

Åström, K. J., & Hägglund, T. (2006). Advanced PID control. ISA - The Instrumentation, Systems, and Automation Society.

Ziegler, J. G., & Nichols, N. B. (1942). Optimum settings for automatic controllers. Transactions of the ASME, 64(11), 759-768.

Åström, K. J., & Hägglund, T. (2001). The future of PID control. Control Engineering Practice, 9(11), 1163-1175.

Seborg, D. E., Edgar, T. F., Mellichamp, D. A., & Doyle, F. J. (2016). Process dynamics and control (4th ed.). Wiley.

Shinskey, F. G. (1996). Process control systems: Application, design, and tuning (4th ed.). McGraw-Hill.